Singleton Design Pattern

The Singleton pattern ensures that a class has only one instance and provides a global access point to that instance.

in simple terms:
"Poore application me sirf ek hi object create hoga."

Real life ex:
1. logging system
2. database connection
3. configuration manager

Key Characteristics:
1.Private constructor → so no one can create object directly
2.Static instance → stores single object
3.Global access method → to get that instance

```
public class Singleton {

    // Step 1: Create static instance
    private static Singleton instance;

    // Step 2: Private constructor
    private Singleton() {}

    // Step 3: Public method to access instance
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton(); // create only once
        }
        return instance;
    }
}
```

Problem with Above (Important)

This is NOT thread-safe.

Agar multiple threads ek saath call kare:

Multiple objects ban sakte hain

Thread-Safe Version (Best Practice)

Double Checked Locking

```
public class Singleton {

    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

When to Use?

Use Singleton when:
You need shared resource
Only one object should exist
Example:
DB connection
Cache
Logger

When NOT to Use?

Avoid when:
You need multiple instances
It makes testing difficult
Tight coupling increases

Eager Loading
Object pehle hi create ho jata hai, jab class load hoti hai

```
class Singleton {
    private static final Singleton instance = new Singleton(); // created immediately

    private Singleton() {}

    public static Singleton getInstance() {
        return instance;
    }
}
```

Lazy Loading
Object tabhi create hota hai jab zarurat ho

```
class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton(); // created on demand
        }
        return instance;
    }
}
```

Eager Loading:

Pros
Simple implementation
Thread-safe by default
No synchronization needed
Faster access (already created)

Cons
Memory waste (agar use hi na ho)
Startup time badh sakta hai
Not good for heavy objects

Lazy Loading

Pros
Memory efficient
Object tabhi banta hai jab needed ho
Better for heavy resources (DB, APIs)

Cons
Thread safety issues
First call me delay
Thoda complex code